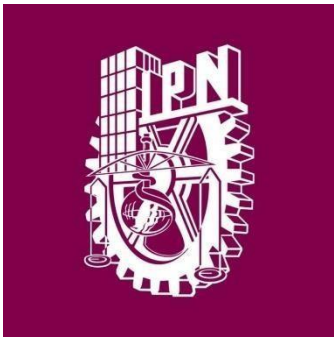




Instituto Politécnico
Nacional
Escuela Superior de
Computo



Profesor: Cortes Galicia Jorge

Materia: Sistemas Operativos

Practica 3

Integrantes:

Vázquez Blancas Cesar Said

Díaz Torres Jonathan Samuel

Aparicio Ponce Roberto Manuel

Corona Hernández Martín Rafael

Introducción Teórica

Los sistemas operativos juegan un papel crucial en la interacción entre el usuario, el hardware y el software, gestionando de manera eficiente los recursos de un sistema informático. Son responsables de controlar y coordinar todo lo que ocurre en una computadora, desde la ejecución de procesos hasta la administración de la memoria, asegurando que múltiples aplicaciones y usuarios puedan operar sin conflictos y de manera ordenada.

Dentro del amplio espectro de sistemas operativos, Windows y Linux son dos de los más conocidos y utilizados. Windows, diseñado con un enfoque comercial, ha sido durante décadas el líder en el ámbito de las computadoras personales. Ofrece una interfaz gráfica de usuario (GUI) intuitiva que facilita su uso a una gran variedad de usuarios, desde principiantes hasta avanzados. Además, incorpora el sistema de archivos NTFS, que brinda soporte para grandes volúmenes de datos, permisos avanzados y opciones de seguridad como la encriptación. Windows también es compatible con una amplia gama de aplicaciones comerciales, lo que lo convierte en una opción ideal para el uso cotidiano en oficinas, hogares y entornos empresariales.



Por otro lado, Linux, basado en el kernel de Unix, es reconocido por su flexibilidad y su capacidad para adaptarse a una gran variedad de entornos. Al ser de código abierto, su evolución ha sido constante gracias a una comunidad activa de desarrolladores que contribuyen a su mejora continua. Linux es conocido por su estabilidad, seguridad y eficiencia, siendo el sistema operativo preferido para servidores, sistemas embebidos y aplicaciones críticas. Aunque cuenta con

interfaces gráficas, su verdadera potencia radica en la línea de comandos (CLI), que permite una personalización detallada y un control profundo sobre el sistema.

Cada sistema operativo tiene características y enfoques que responden a distintas necesidades. Windows se destaca por su facilidad de uso y su amplia compatibilidad con software comercial, mientras que Linux ofrece una mayor flexibilidad, seguridad y robustez, especialmente en entornos donde la estabilidad y el control a nivel del sistema son primordiales.

Además, existen diferentes tipos de sistemas operativos que responden a necesidades específicas. Los sistemas monousuarios, como MS-DOS y versiones antiguas de Windows, están diseñados para gestionar los recursos de un solo usuario. En contraste, los sistemas multiusuario, como Linux y Windows Server, permiten que múltiples usuarios accedan y utilicen los recursos de la misma máquina al mismo tiempo. Los sistemas operativos de tiempo real son esenciales en aplicaciones donde el tiempo de respuesta es crítico, como en sistemas industriales o aeroespaciales. También existen sistemas operativos embebidos, como FreeRTOS y Embedded Linux, que se utilizan en dispositivos específicos, desde electrodomésticos hasta maquinaria industrial.

Finalmente, el trabajo con el lenguaje ensamblador en diferentes plataformas permite comprender mejor cómo interactúan los sistemas operativos con el hardware a nivel fundamental. Este enfoque permite explorar cómo se gestionan las operaciones básicas dentro del sistema, como la manipulación de datos, la ejecución de instrucciones y la gestión de la memoria, proporcionando una perspectiva invaluable para entender los sistemas operativos desde sus cimientos.

Sección Linux:

1. Inicie sesión en Linux.
2. Cree una carpeta llamada Borrار para que allí mantenga sus programas. Al finalizar su sesión de trabajo respalde sus programas en una memoria usb y elimine esta carpeta con su contenido
3. Capture el siguiente código:

```
segment .data ;Segmento de datos
cadena db 'Programando en ensamblador para Linux',0xA ;Cadena a imprimir
segment .text ;Segmento de código
global _start ;Punto de entrada al programa (usado en el enlazador ld)
_start: ;Inicio del programa
    mov edx,38d ;Longitud de cadena
    mov ecx,cadena ;Cadena a escribir
    mov ebx,1 ;Salida estandar
    mov eax,4 ;Numero de llamada al sistema "sys_write"
    int 0x80 ;Interrupción de llamadas al sistema del kernel de Linux
    mov eax,1 ;Número de llamada al sistema "sys_exit"
    int 0x80 ;Interrupción de llamadas al sistema del kernel de Linux
```

4. Guarde su archivo con extensión .asm dentro de Borrار y ensamble desde una consola con el siguiente comando:

`nasm -f elf -o nombre_archivo.o nombre_archivo.asm`

5. Enlace el código objeto intermedio generado en el paso anterior con el siguiente comando:

`ld -m elf_i386 -o nombre_ejecutable nombre_archivo.o`

6. Al final de estos dos pasos obtendrá un programa ejecutable, pruebe el funcionamiento de su aplicación. Dé sus observaciones del código capturado.

```
said@said-VirtualBox:~/Descargas$ nasm -f elf -o hola.o hola.asm
said@said-VirtualBox:~/Descargas$ ld -m elf_i386 -o hola hola.o
said@said-VirtualBox:~/Descargas$ ./hola
Programando en ensamblador para Linux
```

Observaciones Segmento de datos:

- Define una cadena: 'Programando en ensamblador para Linux' con un salto de línea al final (0xA).

Segmento de código:

- Comienza el código en `_start`, que es el punto de entrada al programa.

Escritura de la cadena:

- `mov edx, 38d`: Define la longitud de la cadena (38 caracteres).
- `mov ecx, cadena`: Pasa la dirección de la cadena al registro `ecx`.
- `mov ebx, 1`: Especifica la salida estándar (consola) utilizando el descriptor de archivo 1.
- `mov eax, 4`: Coloca en `eax` el código de la llamada al sistema `sys_write` (número 4).
- `int 0x80`: Realiza la interrupción para ejecutar la llamada al sistema que imprime la cadena.

Finalización del programa:

- `mov eax, 1`: Coloca en `eax` el código de la llamada al sistema `sys_exit` (número 1), que termina el programa.
- `int 0x80`: Realiza la interrupción para finalizar la ejecución del programa.

7. Capture el siguiente código:

```
segment .bss ;Segmento de datos
cadena resb 50 ;Espacio en memoria para la cadena a almacenar
segment .text ;Segmento de código
global _start ;punto de entrada al programa (usado con el enlazador ld)
_start: ;Inicio del programa
    mov edx,50d ;Longitud del bufer
    mov ecx,cadena ;Cadena a leer
    mov ebx,0 ;Entrada estandar
    mov eax,3 ;Numero de llamada al sistema "sys_read"
    int 0x80 ;Interrupción de llamadas al sistema del kernel de Linux
    mov edx,50d ;Longitud de cadena
    mov ecx,cadena ;Cadena a escribir
    mov ebx,1 ;Salida estandar
    mov eax,4 ;Numero de llamada al sistema "sys_write"
    int 0x80 ;Interrupción de llamadas al sistema del kernel de Linux
    mov eax,1 ;Número de llamada al sistema "sys_exit"
    int 0x80 ;Interrupción de llamadas al sistema del kernel de Linux
```

8. Ensamble, enlace y ejecute esta aplicación. Dé sus observaciones del código capturado.

```
said@said-VirtualBox:~/Descargas$ nasm -f elf -o leer.o leer.asm
said@said-VirtualBox:~/Descargas$ ld -m elf_i386 -o leer leer.o
said@said-VirtualBox:~/Descargas$ ./leer
hola
hola
```

Observaciones

- El código reserva un buffer de 50 bytes para almacenar la entrada del usuario.
- Usa la llamada al sistema `sys_read` para capturar lo que el usuario ingrese por la entrada estándar (teclado).
- Posteriormente, utiliza `sys_write` para imprimir la cadena ingresada en la salida estándar (pantalla).
- Tanto la lectura como la escritura están limitadas a 50 bytes, la longitud máxima del buffer.
- El programa finaliza correctamente con `sys_exit`, asegurando que el proceso termine de forma adecuada.
- Es un código simple que combina lectura y escritura usando llamadas al sistema Linux, ideal para entender la interacción con el sistema operativo.

9. Programe una aplicación en ensamblador que genere un contador de 0 a 9, mostrando en pantalla el conteo generado. Consejo: revise las instrucciones de ensamblador CMP, JMP, JE, JNE, e INC.

```
section .data
    valor db 0
    nueva_linea db 10
section .text
global _start
_start:
    mov ebx, 1
cuenta_bucle:
    mov al, [valor]
    add al, '0'
    mov [valor], al
    mov eax, 4
    mov ecx, valor
    mov edx, 1
    int 0x80
    mov eax, 4
    mov ecx, nueva_linea
    mov edx, 1
    int 0x80
    mov al, [valor]
    sub al, '0'
    inc al
    mov [valor], al
    cmp al, 10
    jne cuenta_bucle
fin_programa:
    mov eax, 1
    xor ebx, ebx
    int 0x80
```

```
said@said-VirtualBox:~/Descargas$ nasm -f elf -o contador.o contador.asm
said@said-VirtualBox:~/Descargas$ ld -m elf_i386 -o contador contador.o
said@said-VirtualBox:~/Descargas$ ./contador
```

```
0
1
2
3
4
5
6
7
8
9
```

10. Programe una aplicación en ensamblador que copie 5 cadenas dadas (cadena1 a cadena5) a una nueva cadena (cadena6). La copia de las cadenas deberá ser intercalando los caracteres de cada cadena. Las cadenas dadas deben de ser ingresadas por teclado. Muestre en pantalla el contenido de cadena6. Consejo: revise el uso de los registros índice ESI y EDI.

```
section .data
    prompt_cadena1 db "Ingrese Cadena 1 :", 0
    prompt_cadena2 db "Ingrese Cadena 2 :", 0
    prompt_cadena3 db "Ingrese Cadena 3 :", 0
    prompt_cadena4 db "Ingrese Cadena 4 :", 0
    prompt_cadena5 db "Ingrese Cadena 5 :", 0
    prompt_cadena6 db "Concatenada: |", 0
    len_cadena1 dd 0
    len_cadena2 dd 0
    len_cadena3 dd 0
    len_cadena4 dd 0
    len_cadena5 dd 0
    len_total dd 0
    nl db 0xa

section .bss
    input1 resb 50
    input2 resb 50
    input3 resb 50
    input4 resb 50
    input5 resb 50
    output_concat resb 150

section .text
global _start
_start:
    mov edx, 18
    mov ecx, prompt_cadena1
    mov ebx, 1
    mov eax, 4
    int 0x80
    mov edx, 50
    mov ecx, input1
    mov ebx, 0
    mov eax, 3
    int 0x80
    mov ecx, prompt_cadena2
    mov ebx, 1
    mov eax, 4
    int 0x80
    mov edx, 50
    mov ecx, input2
    mov ebx, 0
    mov eax, 3
    int 0x80
    mov ecx, prompt_cadena3
    mov ebx, 1
    mov eax, 4
    int 0x80
    mov edx, 50
    mov ecx, input3
    mov ebx, 0
    mov eax, 3
    int 0x80
    mov ecx, prompt_cadena4
    mov ebx, 1
    mov eax, 4
    int 0x80
    mov edx, 50
    mov ecx, input4
    mov ebx, 0
    mov eax, 3
    int 0x80
    mov ecx, prompt_cadena5
    mov ebx, 1
    mov eax, 4
    int 0x80
    mov edx, 50
    mov ecx, input5
    mov ebx, 0
    mov eax, 3
    int 0x80
    mov edi, input1
    call strlen
    mov [len_cadena1], eax
    mov edi, input2
    call strlen
    mov [len_cadena2], eax
    add [len_total], eax
    mov edi, input3
    call strlen
    mov [len_cadena3], eax
    add [len_total], eax
    mov edi, input4
    call strlen
    mov [len_cadena4], eax
    add [len_total], eax
    mov edi, input5
    call strlen
    mov [len_cadena5], eax
    add [len_total], eax
    mov eax, 0
    xor edi, edi
    mov edi, output_concat
```

<pre> jmp copy1 copy1: cmp eax, [len_cadena1] jge copy2 mov esi, input1 add esi, eax mov ecx, 1 rep movsb copy2: cmp eax, [len_cadena2] jge copy3 mov esi, input2 add esi, eax mov ecx, 1 rep movsb copy3: cmp eax, [len_cadena3] jge copy4 mov esi, input3 add esi, eax mov ecx, 1 rep movsb copy4: cmp eax, [len_cadena4] jge copy5 mov esi, input4 add esi, eax mov ecx, 1 rep movsb copy5: cmp eax, [len_cadena5] jge end_copy </pre>	<pre> mov esi, input5 add esi, eax mov ecx, 1 rep movsb end_copy: add eax, 1 cmp eax, [len_total] jle copy1 mov edx, 28 mov ecx, prompt_cadena6 mov ebx, 1 mov eax, 4 int 0x80 mov edx, [len_total] mov ecx, output_concat mov ebx, 1 mov eax, 4 int 0x80 mov edx, 1 mov ecx, nl mov ebx, 1 mov eax, 4 int 0x80 mov eax, 1 int 0x80 strlen: mov ebx, edi xor al, al mov ecx, 0xffffffff repne scasb sub edi, ebx sub edi, 2 </pre>	<pre> mov eax, edi ret </pre>
---	--	---

```

said@said-VirtualBox:~/Descargas$ nasm -f elf -o copiacadenas.o copiacadenas.asm
said@said-VirtualBox:~/Descargas$ ld -m elf_i386 -o copiacadenas copiacadenas.o
said@said-VirtualBox:~/Descargas$ ./copiacadenas
Ingrese Cadena 1 :hola
Ingrese Cadena 2 :linux
Ingrese Cadena 3 :sistemas
Ingrese Cadena 4 :operativos
Ingrese Cadena 5 :practica3
Concatenada:
hlsopoiiprlnseautrcxeatmtiaicsvao3s

```

11. Programe una aplicación en ensamblador que muestre en pantalla la longitud de una cadena que haya sido ingresada por teclado. Considere el caso de que la cadena tenga 10 caracteres o más.

```
section .data
    mensaje_entrada db "Ingrese la cadena :", 0
    mensaje_resultado db "Longitud:", 0
section .bss
    buffer_cadena resb 100
    buffer_digitos resb 10
    puntero_digitos resb 8
section .text
global _start
_start:
    mov edx, 20
    mov ecx, mensaje_entrada
    mov ebx, 1
    mov eax, 4
    int 0x80
    mov edx, 100
    mov ecx, buffer_cadena
    mov ebx, 0
    mov eax, 3
    int 0x80
    mov edx, 23
    mov ecx, mensaje_resultado
    mov ebx, 1
    mov eax, 4
    int 0x80
    mov rdi, buffer_cadena
    call calcular_longitud
    sub rax, 1
    call mostrar_digitos
    mov rax, 60
    mov rdi, 0
    syscall

mostrar_digitos:
    mov rcx, buffer_digitos
    mov rbx, 10
    mov [rcx], rbx
    inc rcx
    mov [puntero_digitos], rcx
convertir_a_ascii:
    mov rdx, 0
    mov rbx, 10
    div rbx
    push rax
    add rdx, 48
    mov rcx, [puntero_digitos]
    mov [rcx], dl
    inc rcx
    mov [puntero_digitos], rcx
    pop rax
    cmp rax, 0
    jne convertir_a_ascii
imprimir_digitos:
    mov rcx, [puntero_digitos]
    mov rax, 1
    mov rdi, 1
    mov rsi, rcx
    mov rdx, 1
    syscall
    mov rcx, [puntero_digitos]
    dec rcx
    mov [puntero_digitos], rcx
    cmp rcx, buffer_digitos
    jge imprimir_digitos
    ret

calcular_longitud:
    xor rcx, rcx
calcular_siguiente:
    cmp [rdi], byte 0
    jz fin_calculo
    inc rcx
    inc rdi
    jmp calcular_siguiente
fin_calculo:
    mov rax, rcx
    ret
```

```
said@said-VirtualBox:~/Descargas$ nasm -f elf64 -o long.o long.asm
said@said-VirtualBox:~/Descargas$ ld long.o -o long
said@said-VirtualBox:~/Descargas$ ./long
Ingrese la cadena :hola linux
Longitud:hola linux
10
said@said-VirtualBox:~/Descargas$ ./long
Ingrese la cadena :sistemas operativos
Longitud:sistemas op19
```

12. Programe una aplicación en ensamblador que concatene 5 cadenas (cadena1 a cadena5) ingresadas por teclado, mostrando en pantalla lo siguiente: el contenido de la cadena concatenada, la cadena concatenada en sentido inverso y el número total de vocales que tiene la cadena concatenada.

```

section .data
    msg_instruccion db 'Ingrese 5 cadenas:', 0
    msg_solicitar db 'Ingrese la cadena :', 0
    msg_concatenada db 'Cadena unida :', 0
    msg_inversa db 'Cadena invertida :', 0
    msg_vocales db 'Total vocales :', 0
    salto_linea db 0xa
    buffer_concatenado times 100 db ' '

section .bss
    temp_char resb 1
    longitud resb 1
    cadena_actual resb 50
    buffer_inverso resb 500
    contador_vocales resd 1
    num_cadenas resb 8
    digitos_vocales resb 100

section .text
    global _start
    _start:
        mov edx, 18
        mov ecx, msg_instruccion
        mov ebx, 1
        mov eax, 4
        int 0x80
        mov edx, 1
        mov ecx, salto_linea
        mov ebx, 1
        mov eax, 4
        int 0x80
        mov edx, 19
        mov ecx, msg_solicitar
        mov ebx, 1
        mov eax, 4
        int 0x80
        mov esi, cadena_actual
        call invertir_cadena
        mov edx, 18
        mov edi, buffer_concatenado
        mov edx, 50
        mov ecx, cadena_actual
        mov ebx, 0
        mov eax, 3
        int 0x80
        mov esi, cadena_actual
        call encontrar_final
        times 4 call pedir_cadena
        mov rdi, buffer_concatenado
        call calcular_longitud
        mov [longitud], rax
        mov [temp_char], rax
        push temp_char
        mov edx, 15
        mov ecx, msg_concatenada
        mov ebx, 1
        mov eax, 4
        int 0x80
        mov edx, [temp_char]
        mov ecx, buffer_concatenado
        mov ebx, 1
        mov eax, 4
        int 0x80
        mov esi, cadena_actual
        mov ebx, 0
        mov eax, 3
        int 0x80
        mov esi, cadena_actual
        inc edi

mov ecx, msg_inversa
mov ebx, 1
mov eax, 4
int 0x80
mov edx, [temp_char]
mov ecx, buffer_inverso
mov ebx, 1
mov eax, 4
int 0x80
mov esi, contador_vocales
mov ebx, 1
mov eax, 4
int 0x80
mov [digitos_vocales], eax
call imprimir_numero
mov eax, 1
int 0x80

pedir_cadena:
    mov edx, 19
    mov ecx, msg_solicitar
    mov ebx, 1
    mov eax, 4
    int 0x80
    mov edx, 50
    mov ecx, cadena_actual
    mov ebx, 0
    mov eax, 3
    int 0x80
    mov esi, cadena_actual
    inc edi

```

```

encontrar_final:
    cmp [esi], byte 0xa
    jz fin_lectura
    mov al, [esi]
    cmp al, 'a'
    je contar_vocal
    cmp al, 'e'
    je contar_vocal
    cmp al, 'i'
    je contar_vocal
    cmp al, 'o'
    je contar_vocal
    cmp al, 'u'
    je contar_vocal
    jmp siguiente_char

contar_vocal:
    add dword [contador_vocales], 1
siguiente_char:
    mov ecx, 1
    rep movsb
    jmp encontrar_final

calcular_longitud:
    xor rcx, rcx
recorrer:
    cmp [rdi], byte ' '
    jz finalizar
    inc rcx
    inc rdi
    jmp recorrer

finalizar:
    mov rax, rcx
    ret

imprimir_numero:
    mov rcx, digitos_vocales
    mov rbx, 10
    mov [rcx], rbx
    inc rcx
    mov [num_cadenas], rcx
imprimir_bucle:
    mov rdx, 0
    mov rbx, 10
    div rbx
    push rax
    add rdx, 48
    mov rcx, [num_cadenas]
    mov [rcx], dl
    inc rcx
    mov [num_cadenas], rcx
    pop rax
    cmp rax, 0
    jne imprimir_bucle

imprimir_final:
    mov rcx, [num_cadenas]
    mov rax, 1
    mov rdi, 1
    mov rsi, rcx
    mov rdx, 1
    syscall
    mov rcx, [num_cadenas]
    dec rcx
    mov [num_cadenas], rcx
    cmp rcx, digitos_vocales
    jge imprimir_final
    ret

invertir_cadena:
    mov esi, buffer_concatenado
    sub esi, 1
invertir_bucle:
    cmp [esi], byte 0
    jz fin_inversion
    mov edi, buffer_inverso
    add edi, [temp_char]
    mov ecx, 1
    rep movsb
    sub byte [temp_char], 1
    jmp invertir_bucle

fin_inversion:
    ret
fin_lectura:
    sub edi, 1
    ret

```

```
said@said-VirtualBox:~/Descargas$ nasm -f elf64 -o inversa.o inversa.asm
said@said-VirtualBox:~/Descargas$ ld inversa.o -o inversa
said@said-VirtualBox:~/Descargas$ ./inversa
Ingrese 5 cadenas:
Ingrese la cadena :a
Ingrese la cadena :i
Ingrese la cadena :k
Ingrese la cadena :f
Ingrese la cadena :l
Cadena unida :aikfl
Cadena invertida :lfkia
Total vocales :
2
said@said-VirtualBox:~/Descargas$ ./inversa
Ingrese 5 cadenas:
Ingrese la cadena :i
Ingrese la cadena :h
Ingrese la cadena :d
Ingrese la cadena :k
Ingrese la cadena :l
Cadena unida :ihdkl
Cadena invertida :lkdhi
Total vocales :
1
```

13. Programe una aplicación en ensamblador que implemente una calculadora con las cuatro operaciones básicas. A través de un menú, dé la posibilidad de seleccionar la operación a realizar. Maneje únicamente dígitos enteros positivos en el intervalo [0, 255]. Consejo: revise las instrucciones de ensamblador ADD, SUB, MUL, y DIV.

```

section .data
    opc1 db '1.- Suma',0xa
    opc2 db '2.- Resta',0xa
    opc3 db '3.- Division',0xa
    opc4 db '4.- Multiplicacion',0xa
    opc db 'Eleccion:',0
    opc1 equ ($-opc)
    soln1 db 'Digito 1 :',0
    soln2 db 'Digito 2 :',0
    nega db '-'
    negal equ ($-nega)
    suma db 'Suma',10
    sumal equ ($-suma)
    resu db 'Resultado :',0
section .bss
    opcion resb 2
    num1 resb 50
    num2 resb 50
    espDigito resb 100
    posDigito resb 8
section .text
global _start
_start:
    mov edx, 9
    mov ecx, opc1
    mov ebx, 1
    mov eax, 4
    int 0x80
    mov edx, 10
    mov ecx, opc2
    mov ebx, 1
    mov eax, 4
    int 0x80
    mov edx, 13
    mov ecx, opc3
    mov ebx, 1
    mov eax, 4
    int 0x80
    mov edx, 19
    mov ecx, opc4
    mov ebx, 1
    mov eax, 4
    int 0x80
    mov edx, 10
    mov ecx, opc
    mov ebx, 1
    mov eax, 4
    int 0x80
    mov edx, 2
    mov ecx, opcion
    mov ebx, 0
    mov eax, 3
    int 0x80
    mov edx, 11
    mov ecx, soln1
    mov ebx, 1
    mov eax, 4
    int 0x80
    mov edx, 50
    mov ecx, num1
    mov ebx, 0
    mov eax, 3
    int 0x80
    mov edx, 11
    mov ecx, soln2
    mov ebx, 1
    mov eax, 4
    int 0x80
    mov ebx, 1
    mov eax, 4
    int 0x80
    mov ecx, num2
    mov ebx, 0
    mov eax, 3
    int 0x80
    sub byte [opcion], '0'
    cmp byte [opcion], byte 1
    je sum
    cmp byte [opcion], byte 2
    je res
    cmp byte [opcion], byte 3
    je divi
    cmp byte [opcion], byte 4
    je multi
sum:
    mov edx, 13
    mov ecx, resu
    mov ebx, 1
    mov eax, 4
    int 0x80
    mov rax, num1
    call atoi
    push rax
    mov rax, num2
    call atoi
    pop rdx
    add rax, rdx
    call imprimir
    mov eax, 1

```

```

    int 0x80
res:
    mov edx, 13
    mov ecx, resu
    mov ebx, 1
    mov eax, 4
    int 0x80
    mov rax, num1
    call atoi
    push rax
    mov rax, num2
    call atoi
    pop rdx
    cmp rax, rdx
    jg cambio
    sub rdx, rax
    mov rax, rdx
    call imprimir
    mov eax, 1
    int 0x80
cambio:
    push rax
    push rdx
    mov edx, negal
    mov ecx, nega
    mov ebx, 1
    mov eax, 4
    int 0x80
    pop rdx
    pop rax
    sub rax, rdx
    call imprimir

    mov eax, 1
    int 0x80
divi:
    mov edx, 13
    mov ecx, resu
    mov ebx, 1
    mov eax, 4
    int 0x80
    mov rax, num1
    call atoi
    push rax
    mov rax, num2
    call atoi
    pop rdx
    mov rax, rax
    pop rax
    div rbx
    call imprimir
    mov eax, 1
    int 0x80
multi:
    mov edx, 13
    mov ecx, resu
    mov ebx, 1
    mov eax, 4
    int 0x80
    mov rax, num1
    call atoi
    push rax
    mov rax, num2
    call atoi
    mov rax, rax
    pop rax

    mul rbx
    call imprimir
    mov eax, 1
    int 0x80
atoi:
    mov esi, eax
    mov eax, 0
    mov ecx, 0
.potencia10:
    xor ebx, ebx
    mov bl, [esi+ecx]
    cmp bl, 48
    jl .finatoi
    jg .finatoi
    cmp bl, 57
    jg .finatoi
    cmp bl, 10
    je .finatoi
    cmp bl, 0
    jz .finatoi
    sub bl, 48
    add eax, ebx
    mov ebx, 10
    mul ebx
    inc ecx
    jmp .potencia10
.finatoi:
    mov ebx, 10
    div ebx
    ret
imprimir:
    mov rcx, espDigito
    mov rbx, 10

    mov [rcx], rbx
    inc rcx
    mov [posDigito], rcx
bucle:
    mov rdx, 0
    mov rbx, 10
    div rbx
    push rax
    add rdx, 48
    mov rcx, [posDigito]
    mov [rcx], dl
    inc rcx
    mov [posDigito], rcx
    pop rax
    cmp rax, 0
    jne bucle
bucle2:
    mov rcx, [posDigito]
    mov rax, 1
    mov rdi, 1
    mov rsi, rcx
    mov rdx, 1
    syscall
    mov rcx, [posDigito]
    dec rcx
    mov [posDigito], rcx
    cmp rcx, espDigito
    jge bucle2
    ret

```

```
said@said-VirtualBox:~/Descargas$ rm calculadora.asm calculadora.o calculadora
said@said-VirtualBox:~/Descargas$ nasm -f elf64 -o calculadora.o calculadora.asm
said@said-VirtualBox:~/Descargas$ ld calculadora.o -o calculadora
said@said-VirtualBox:~/Descargas$ ./calculadora
1.- Suma
2.- Resta
3.- Division
4.- Multiplicacion
Eleccion:1
Digito 1 :5
Digito 2 :8
Resultado :13
said@said-VirtualBox:~/Descargas$ ./calculadora
1.- Suma
2.- Resta
3.- Division
4.- Multiplicacion
Eleccion:2
Digito 1 :6
Digito 2 :9
Resultado :-3
said@said-VirtualBox:~/Descargas$ ./calculadora
1.- Suma
2.- Resta
3.- Division
4.- Multiplicacion
Eleccion:3
Digito 1 :15
Digito 2 :5
Resultado :3
```

```
Resultado :3
said@said-VirtualBox:~/Descargas$ ./calculadora
1.- Suma
2.- Resta
3.- Division
4.- Multiplicacion
Eleccion:4
Digito 1 :5
Digito 2 :9
Resultado :45
```

Sección Windows:

1. Inicie sesión en Windows.

2. Cree una carpeta llamada **Borrar en disco duro** para que allí mantenga sus programas. Al finalizar su sesión de trabajo elimine esta carpeta con su contenido.

3. Capture el siguiente código:

```
segment .data
    cadImprimir db 'Ensamblando en Windows',0xA ;Segundo argumento para la llamada al sistema _WriteConsoleA()
segment .bss
    handleConsola resd 1 ;Primer argumento para la llamada al sistema _WriteConsoleA()
    longitudCadena resd 1 ;Tercer argumento para la llamada al sistema _WriteConsoleA()
    caractEscritos resd 1 ;Cuarto argumento para la llamada al sistema _WriteConsoleA()
    ultimoArgumento resd 1 ;Quinto argumento para la llamada al sistema _WriteConsoleA()
segment .text
global _main
extern _GetStdHandle@4 ;Acceso externo a la llamada al sistema _GetStdHandle()
extern _WriteConsoleA@20 ;Acceso externo a la llamada al sistema _WriteConsoleA()
extern _ExitProcess@4 ;Acceso externo a la llamada al sistema _ExitProcess()
_main: push dword -11 ;Argumento pasado por la pila y usado en _GetStdHandle() para la salida estándar
    call _GetStdHandle@4 ;Invocacion de _GetStdHandle()
    mov [handleConsola],eax ;Devolucion del manejador de consola para escritura en el registro eax
    xor eax,eax ;limpieza del registro eax (eax=0)
    mov eax,23d ;eax=23 caracteres de longitud de la cadena a imprimir
    mov [longitudCadena],eax ;Se guarda la longitud en memoria
    xor eax,eax ;limpieza del registro eax (eax=0)
    mov eax,0d ;eax=0 valor del ultimo argumento de _WriteConsoleA()
    mov [ultimoArgumento],eax ;Se guarda el valor del ultimo argumento en memoria
    push dword [ultimoArgumento] ;Quinto argumento de _WriteConsoleA() pasado por la pila
    push dword caractEscritos ;Cuarto argumento de _WriteConsoleA() pasado por la pila
    push dword [longitudCadena] ;Tercer argumento de _WriteConsoleA() pasado por la pila
    push dword cadImprimir ;Segundo argumento de _WriteConsoleA() pasado por la pila
    push dword [handleConsola] ;Primer argumento de _WriteConsoleA() pasado por la pila
    call _WriteConsoleA@20 ;Invocacion de _WriteConsoleA()
    xor eax,eax ;limpieza del registro eax (eax=0)
    mov eax,0d ;eax=0 valor del argumento de _ExitProcess()
    mov [ultimoArgumento],eax ;Se guarda el valor del argumento en memoria
    push dword [ultimoArgumento] ;Argumento de _ExitProcess() pasado por la pila
    call _ExitProcess@4 ;Invocacion de _ExitProcess()
```

4. Guarde su archivo con extensión **.asm** dentro de **Borrar** y ensamble desde una consola con el siguiente comando: **nasm -f win32 -o nombre_archivo.obj nombre_archivo.asm**

5. Enlace el código objeto intermedio generado en el paso anterior con el siguiente comando: **ld nombre_archivo.obj -m i386pe -e _main -L "ruta_al_archvivo_kernel32" -l kernel32 -o nombre_ejecutable.exe**

6. Al final de estos dos pasos obtendrá un programa ejecutable, pruebe el funcionamiento de su aplicación. Dé sus observaciones del código capturado.

```
C:\Users\vbcs1>nasm -f win32 -o leerw.obj leerw.asm

C:\Users\vbcs1>ld leerw.obj -m i386pe -e _main -L "C:\Windows\SysWOW64" -l kernel32 -o leer.exe
ld: warning: resolving _GetStdHandle@4 by linking to _GetStdHandle
Use --enable-stdcall-fixup to disable these warnings
Use --disable-stdcall-fixup to disable these fixups
ld: warning: resolving _WriteConsoleA@20 by linking to _WriteConsoleA
ld: warning: resolving _ExitProcess@4 by linking to _ExitProcess

C:\Users\vbcs1>leer.exe
Ensamblando en Windows
```

Observaciones

- El programa está diseñado para ejecutarse en Windows, interactuando con la consola a través de las funciones **GetStdHandle** y **WriteConsoleA**.
- Usa memoria reservada para almacenar los distintos parámetros que se pasarán a **WriteConsoleA**, tales como el manejador de la consola y el número de caracteres escritos.
- Obtiene el manejador de la consola usando **GetStdHandle**, lo cual permite escribir en la salida estándar (la consola).
- Calcula y almacena la longitud de la cadena a imprimir, que es "Ensamblando en Windows".
- Mediante el uso de la pila, se pasan los argumentos necesarios a **WriteConsoleA** para escribir la cadena en la consola.
- Después de escribir en la consola, el programa realiza una llamada a **ExitProcess** para finalizar correctamente el proceso.
- Todo el manejo de las funciones de Windows se realiza de manera ordenada, utilizando registros y la pila para preparar los argumentos de las llamadas al sistema.

7. Capture el siguiente código:

```
segment .bss
handleConsola resd 1 ;Primer argumento para la llamada al sistema _ReadConsoleA()
cadLeer resb 30 ;Segundo argumento para la llamada al sistema _ReadConsoleA()
longitudCadena resd 1 ;Tercer argumento para la llamada al sistema _ReadConsoleA()
caractLeidos resd 1 ;Cuarto argumento para la llamada al sistema _ReadConsoleA()
ultimoArgumento resd 1 ;Quinto argumento para la llamada al sistema _ReadConsoleA()
segment .text
global _main
extern _GetStdHandle@4 ;Acceso externo a la llamada al sistema _GetStdHandle()
extern _WriteConsoleA@20 ;Acceso externo a la llamada al sistema _WriteConsoleA()
extern _ReadConsoleA@20 ;Acceso externo a la llamada al sistema _ReadConsoleA()
extern _ExitProcess@4 ;Acceso externo a la llamada al sistema _ExitProcess()
_main: push dword -10 ;Argumento pasado por la pila y usado en _GetStdHandle() para la entrada estandar
call _GetStdHandle@4 ;Invocacion de _GetStdHandle()
mov [handleConsola],eax ;Devolucion del manejador de consola para lectura en el registro eax
xor eax,eax ;Limpieza del registro eax (eax=0)
mov eax,30d ;eax=30 caracteres de longitud de la cadena a leer
mov [longitudCadena],eax ;Se guarda la longitud en memoria
xor eax,eax ;Limpieza del registro eax (eax=0)
mov eax,0d ;eax=0 valor del ultimo argumento de _ReadConsoleA()
mov [ultimoArgumento],eax ;Se guarda el valor del ultimo argumento en memoria
push dword [ultimoArgumento] ;Quinto argumento de _ReadConsoleA() pasado por la pila
push dword caractLeidos ;Cuarto argumento de _ReadConsoleA() pasado por la pila
push dword [longitudCadena] ;Tercer argumento de _ReadConsoleA() pasado por la pila
push dword cadLeer ;Segundo argumento de _ReadConsoleA() pasado por la pila
push dword [handleConsola] ;Primer argumento de _ReadConsoleA() pasado por la pila
call _ReadConsoleA@20 ;Invocacion de _ReadConsoleA()
xor eax,eax
push dword -11 ;Argumento pasado por la pila y usado en _GetStdHandle() para la salida estandar
call _GetStdHandle@4 ;Invocacion de _GetStdHandle()
mov [handleConsola],eax ;Devolucion del manejador de consola para escritura en el registro eax
xor eax,eax ;Limpieza del registro eax (eax=0)
mov eax,30d ;eax=30 caracteres de longitud de la cadena a imprimir
```

```

mov [longitudCadena],eax ;Se guarda la longitud en memoria
xor eax,eax ;Limpieza del registro eax (eax=0)
mov eax,0d ;eax=0 valor del ultimo argumento de _WriteConsoleA()
mov [ultimoArgumento],eax ;Se guarda el valor del ultimo argumento en memoria
push dword [ultimoArgumento] ;Quinto argumento de _WriteConsoleA() pasado por la pila
push dword caractLeidos ;Cuarto argumento de _WriteConsoleA() pasado por la pila
push dword [longitudCadena] ;Tercer argumento de _WriteConsoleA() pasado por la pila
push dword cadLeer ;Segundo argumento de _WriteConsoleA() pasado por la pila
push dword [handleConsola] ;Primer argumento de _WriteConsoleA() pasado por la pila
call _WriteConsoleA@20 ;Invocacion de _WriteConsoleA()
xor eax,eax ;Limpieza del registro eax (eax=0)
mov eax,0d ;eax=0 valor del argumento de _ExitProcess()
mov [ultimoArgumento],eax ;Se guarda el valor del argumento en memoria
push dword [ultimoArgumento] ;Argumento de _ExitProcess() pasado por la pila
call ExitProcess@4 ;Invocacion de ExitProcess()

```

8. Ensamble (ignore la advertencia generada en caso de que se muestre), enlace y ejecute. Dé sus observaciones acerca del funcionamiento del programa.

```

C:\Users\vbcsl>nasm -f win32 -o lecturaw.obj lecturaw.asm

C:\Users\vbcsl>ld lecturaw.obj -m i386pe -e _main -L "C:\Windows\SysWOW64" -l kernel32 -o lectura.exe
ld: warning: resolving _GetStdHandle@4 by linking to _GetStdHandle
Use --enable-stdcall-fixup to disable these warnings
Use --disable-stdcall-fixup to disable these fixups
ld: warning: resolving _WriteConsoleA@20 by linking to _WriteConsoleA
ld: warning: resolving _ReadConsoleA@20 by linking to _ReadConsoleA
ld: warning: resolving _ExitProcess@4 by linking to _ExitProcess

C:\Users\vbcsl>lectura.exe
hola windows
hola windows

```

Observaciones

- **Lectura desde la consola:** El código utiliza ReadConsoleA para leer hasta 30 caracteres de la entrada estándar (teclado). Esto es similar a cómo se usó sys_read en Linux, pero con funciones específicas de Windows.
- **Uso de la pila:** Los argumentos para la llamada a ReadConsoleA y WriteConsoleA se pasan utilizando la pila. Esto incluye el manejador de la consola, el buffer de lectura, la longitud y el número de caracteres leídos.
- **Manejador de consola:** Se obtiene un manejador de la consola con GetStdHandle, tanto para la entrada estándar (lectura) como para la salida estándar (escritura). Este manejador se almacena en memoria y se usa posteriormente en las funciones de lectura y escritura.
- **Escritura en la consola:** Después de leer la cadena desde la entrada estándar, se utiliza WriteConsoleA para imprimir esa cadena en la salida estándar (pantalla).

9. Programe las aplicaciones de los puntos 9 al 13 de la sección de Linux usando el ensamblador para Windows.

contadorw.asm

```
segment .data
    valor db 0
    nueva_linea db 13, 10
    mensaje db 'Numero: ', 0
    hStdOut dd 0
segment .bss
segment .text
global _main
extern _GetStdHandle@4
extern _WriteConsoleA@20
extern _ExitProcess@4
_main:
    push -11
    call _GetStdHandle@4
    mov [hStdOut], eax
    mov ecx, 1
cuenta_bucle:
    mov eax, [hStdOut]
    push 0
    push 7
    push mensaje
    push eax
    call _WriteConsoleA@20
    mov al, [valor]
    add al, '0'
    mov [valor], al
    mov eax, [hStdOut]
    push 0
    push 1
    push valor
    push eax
    call _WriteConsoleA@20
    mov eax, [hStdOut]
    push 0
    push 2
    push nueva_linea
    push eax
    call _WriteConsoleA@20
    mov al, [valor]
    sub al, '0'
    inc al
    mov [valor], al
    cmp al, 10
    jne cuenta_bucle
fin_programa:
    push 0
    call _ExitProcess@4
```

```
C:\Users\vbcsl>nasm -f win32 -o contadorw.obj contadorw.asm
C:\Users\vbcsl>ld contadorw.obj -m i386pe -e _main -L "C:\Windows\SysWOW64" -l kernel32 -o contador.exe
ld: warning: resolving _GetStdHandle@4 by linking to _GetStdHandle
Use --enable-stdcall-fixup to disable these warnings
Use --disable-stdcall-fixup to disable these fixups
ld: warning: resolving _WriteConsoleA@20 by linking to _WriteConsoleA
ld: warning: resolving _ExitProcess@4 by linking to _ExitProcess

C:\Users\vbcsl>contador.exe
Numero:0
Numero:1
Numero:2
Numero:3
Numero:4
Numero:5
Numero:6
Numero:7
Numero:8
Numero:9
```

[illegible]

```
push dword [handleCon]
call _ReadConsoleA@20
push dword -11
call _GetStdHandle@4
mov [handleCon], eax
mov eax, 0
mov [parametroFinal], eax
push dword [parametroFinal]
push dword charsEscritos
push dword 11
push dword mensaje4
push dword [handleCon]
call _WriteConsoleA@20
push dword -10
call _GetStdHandle@4
mov [handleCon], eax
mov eax, 50
mov [tamanoCadena], eax
push dword [parametroFinal]
push dword charsEscritos
push dword [tamanoCadena]
push dword cadenaE
push dword [handleCon]
call _ReadConsoleA@20
mov edi, cadenaA
call _calcLength
mov [longitudA], eax
mov [longitudFinal], eax
mov edi, cadenaB
call _calcLength
mov [longitudB], eax
add [longitudFinal], eax
mov edi, cadenaC
call _calcLength
mov [longitudC], eax
add [longitudFinal], eax
mov edi, cadenaD
call _calcLength
mov [longitudD], eax
add [longitudFinal], eax
mov edi, cadenaE
call _calcLength
mov [longitudE], eax
add [longitudFinal], eax
xor eax, eax
mov edi, cadenaFinal
jmp unirA

unirA:
cmp eax, [longitudA]
jge unirB
mov esi, cadenaA
add esi, eax
mov ecx, 1
rep movsb

unirB:
cmp eax, [longitudB]
jge unirC
mov esi, cadenaB
add esi, eax
mov ecx, 1
rep movsb

unirC:
cmp eax, [longitudC]
jge unirD
mov esi, cadenaC
add esi, eax
mov ecx, 1
rep movsb

unirD:
cmp eax, [longitudD]
jge unirE
mov esi, cadenaD
add esi, eax
mov ecx, 1
```

```

    rep movsb
unirE:
    cmp eax, [longitudE]
    jge finConcatenacion
    mov esi, cadenaE
    add esi, eax
    mov ecx, 1
    rep movsb
    jmp finConcatenacion
finConcatenacion:
    add eax, 1
    cmp eax, [longitudFinal]
    jle unirA
    jmp fin
fin:
    push dword -11
    call _GetStdHandle@4
    mov [handleCon], eax
    xor eax, eax
    mov eax, 0
    mov [parametroFinal], eax
    push dword [parametroFinal]
    push dword charsEscritos
    push dword 25
    push dword resultado
    push dword [handleCon]
    call _WriteConsoleA@20
    push dword -11
    call _GetStdHandle@4
    mov [handleCon], eax
    push dword [parametroFinal]
    push dword charsEscritos
    push dword [longitudFinal]
    push dword cadenaFinal
    push dword [handleCon]
    call _WriteConsoleA@20
    xor eax, eax
    push dword [parametroFinal]
    call _ExitProcess@4
_calcLength:
    mov ebx, edi
    xor al, al
    mov ecx, 0xffffffff
    repne scasb
    sub edi, ebx
    sub edi, 1
    mov eax, edi
    ret

```

```

C:\Users\vbcs1>copla.exe
Cadena 1 :hola
Cadena 2 :windows
Cadena 3 :sistemas
Cadena 4 :operativos
Cadena 5 :practica3
oeatltado (intercalado)>hwsopoiiprlnseaadtroc
svaisaic
o3
s

```

longw.asm

```
segment .data
    mensaje_entrada db 'Ingresa una cadena: ', 0
    mensaje_longitud db 'La longitud de la cadena es: ', 0
segment .bss
    espDigito resd 100
    handleConsola resd 1
    caracteresEscritos resd 1
    tamanoCadena resd 1
    argumentoFinal resd 1
    bufferCadena resb 100
    punteroDigito resd 1
segment .text
global _main
extern _GetStdHandle@4
extern _ReadConsoleA@20
extern _WriteConsoleA@20
extern _ExitProcess@4
_main:
    push dword -11
    call _GetStdHandle@4
    mov [handleConsola], eax
    xor eax, eax
    mov eax, 0d
    mov [argumentoFinal], eax
    push dword [argumentoFinal]
    push dword caracteresEscritos
    push dword 20
    push dword mensaje_entrada
    push dword [handleConsola]
    call _WriteConsoleA@20
    push dword -10
    call _GetStdHandle@4
    mov [handleConsola], eax
    xor eax, eax
    mov eax, 100d
    mov [tamanoCadena], eax
    xor eax, eax
    mov eax, 0d
    mov [argumentoFinal], eax
    push dword [argumentoFinal]
    push dword caracteresEscritos
    push dword [tamanoCadena]
    push dword bufferCadena
    push dword [handleConsola]
    call _ReadConsoleA@20
    push dword -11
    call _GetStdHandle@4
    mov [handleConsola], eax
    xor eax, eax
    mov eax, 0d
    mov [argumentoFinal], eax
    push dword [argumentoFinal]
    push dword caracteresEscritos
    push dword 28
    push dword mensaje_longitud
    push dword [handleConsola]
    call _WriteConsoleA@20
    mov edi, bufferCadena
    call calcular_longitud
    sub eax, 2
    call imprimir_resultado
    xor eax, eax
    mov eax, 0d
    mov [argumentoFinal], eax
    push dword [argumentoFinal]
    call _ExitProcess@4
    imprimir_resultado:
    mov ecx, espDigito
    mov ebx, 10
    mov [ecx], ebx
    inc ecx
    mov [punteroDigito], ecx
convertir_a_ascii:
    mov edx, 0
    mov ebx, 10
    div ebx
    add edx, 48
    mov ecx, [punteroDigito]
    mov [ecx], dl
    inc ecx
    mov [punteroDigito], ecx
    cmp eax, 0
    jne convertir_a_ascii
imprimir_digitos:
    mov ecx, [punteroDigito]
    push dword -11
    xor eax, eax
    call _GetStdHandle@4
    mov [handleConsola], eax
    xor eax, eax
    mov eax, 0d
    mov [argumentoFinal], eax
    push dword [argumentoFinal]
    push dword caracteresEscritos
    push dword 1
    push dword ecx
    push dword [handleConsola]
```

```
    call _WriteConsoleA@20;
    mov ecx, [punteroDigito]
    dec ecx
    mov [punteroDigito], ecx
    cmp ecx, espDigito
    jge imprimir_digitos
    ret
calcular_longitud:
    xor ecx, ecx
contar_caracteres:
    cmp [edi], byte 0
    je fin
    inc ecx
    inc edi
    jmp contar_caracteres
fin:
    mov eax, ecx
    ret
```

```
C:\Users\vbcsl>nasm -f win32 -o longw.obj longw.asm

C:\Users\vbcsl>ld longw.obj -m i386pe -e _main -L "C:\Windows\SysWOW64" -l kernel32 -o long.exe
ld: warning: resolving _GetStdHandle@4 by linking to _GetStdHandle
Use --enable-stdcall-fixup to disable these warnings
Use --disable-stdcall-fixup to disable these fixups
ld: warning: resolving _ReadConsoleA@20 by linking to _ReadConsoleA
ld: warning: resolving _WriteConsoleA@20 by linking to _WriteConsoleA
ld: warning: resolving _ExitProcess@4 by linking to _ExitProcess

C:\Users\vbcsl>long.exe
Ingresa una cadena: hola windows
La longitud de la cadena es:12
```

Inversaw.asm

```
segment .data
    instruccion db 'Digite 5 cadenas:',0
    mensaje db 'Cadena :',0
    concat db 'Cadena concatenada :',0
    inver db 'Cadena inversa :',0
    voca db 'Cantidad vocales :',0
    salto db 0xD, 0xA
    i db 0
    longitud dd 0
    vocales dd 0
segment .bss
    handleConsola resd 1
    caractEscritos resd 1
    longitudCadena resd 1
    ultimoArgumento resd 1
    cadena resb 101
    resultado resb 1000
    reversa resb 1000
    longitudStr resb 4
    vocalesStr resb 4
segment .text
global _main
extern _GetStdHandle@4
extern _WriteConsoleA@20
extern _ReadConsoleA@20
extern _ExitProcess@4
_main:
    mov edi, resultado
    mov byte[i], 0
    mov dword[longitud], 0
    push dword -11
    xor eax, eax
    call _GetStdHandle@4
    mov [handleConsola], eax
    xor eax, eax
    mov eax, 0d
    mov [ultimoArgumento], eax
    push dword [ultimoArgumento]
    push dword caractEscritos
    push dword 18
    push dword instruccion
    push dword [handleConsola]
    call _WriteConsoleA@20
    push dword -11
    xor eax, eax
    call _GetStdHandle@4
    mov [handleConsola], eax
    xor eax, eax
    mov eax, 0d
    mov [ultimoArgumento], eax
    push dword [ultimoArgumento]
    push dword caractEscritos
    push dword 2
    push dword salto
    push dword [handleConsola]
    call _WriteConsoleA@20
    pedirCadenas:
    push dword -11
    xor eax, eax
    call _GetStdHandle@4
    mov [handleConsola], eax
    xor eax, eax
    mov eax, 0d
    mov [ultimoArgumento], eax
    push dword [ultimoArgumento]
```

```
    push dword caractEscritos
    push dword 9
    push dword mensaje
    push dword [handleConsola]
    call _WriteConsoleA@20
    push dword -10
    call _GetStdHandle@4
    mov [handleConsola], eax
    xor eax, eax
    mov eax, 2
    mov [longitudCadena], eax
    xor eax, eax
    mov eax, 0d
    mov [ultimoArgumento], eax
    push dword [ultimoArgumento]
    push dword caractEscritos
    push dword 101
    push dword cadena
    push dword [handleConsola]
    call _ReadConsoleA@20
    mov esi, cadena
    call copiarCadena
    inc byte[i]
    cmp byte[i], 5
    je imprimirResultado
    jne pedirCadenas
copiarCadena:
    mov dl, byte[esi]
    mov byte[edi], dl
    inc edi
    inc esi
    inc dword[longitud]
    esVocal:
    cmp dl, 'a'
    je incrementarVocales
    cmp dl, 'e'
    je incrementarVocales
    cmp dl, 'i'
    je incrementarVocales
    cmp dl, 'o'
    je incrementarVocales
    cmp dl, 'u'
    je incrementarVocales
    cmp dl, 'A'
    je incrementarVocales
    cmp dl, 'E'
    je incrementarVocales
    cmp dl, 'I'
    je incrementarVocales
    cmp dl, 'O'
    je incrementarVocales
    cmp dl, 'U'
    je incrementarVocales
    ret
incrementarVocales:
    inc dword[vocales]
    ret
imprimirResultado:
    push dword -11
    xor eax, eax
    call _GetStdHandle@4
    mov [handleConsola], eax
    xor eax, eax
    mov eax, 0d
    mov [ultimoArgumento], eax
    push dword [ultimoArgumento]
    call _GetStdHandle@4
    mov [handleConsola], eax
    xor eax, eax
    mov eax, 0d
    mov [ultimoArgumento], eax
    push dword [ultimoArgumento]
    push dword caractEscritos
    push dword 21
    push dword concat
    push dword [handleConsola]
    call _WriteConsoleA@20
    push dword -11
    xor eax, eax
    call _GetStdHandle@4
    mov [handleConsola], eax
    xor eax, eax
    mov eax, 0d
    mov [ultimoArgumento], eax
    push dword [ultimoArgumento]
    push dword caractEscritos
    push dword [longitud]
    push dword resultado
    push dword [handleConsola]
    call _WriteConsoleA@20
    push dword -11
    xor eax, eax
    call _GetStdHandle@4
    mov [handleConsola], eax
    xor eax, eax
    mov eax, 0d
    mov [ultimoArgumento], eax
    push dword [ultimoArgumento]
```

```

push dword caractEscritos
push dword 2
push dword salto
push dword [handleConsola]
call _WriteConsoleA@20
mov esi, reversa
dec edi
invertir:
mov dl, byte[edi]
mov byte[esi], dl
dec edi
inc esi
cmp edi, resultado
j1l imprimirReversa
jge invertir
ret
imprimirReversa:
push dword-11
xor eax, eax
call _GetStdHandle@4
mov [handleConsola], eax
xor eax, eax
mov eax, 0d
mov [ultimoArgumento], eax
push dword [ultimoArgumento], eax
push dword caractEscritos
push dword 17
push dword inver
push dword [handleConsola]
call _WriteConsoleA@20
push dword-11
xor eax, eax
call _GetStdHandle@4
mov [handleConsola], eax
xor eax, eax
mov eax, 0d
mov [ultimoArgumento], eax
push dword [ultimoArgumento], eax
push dword caractEscritos
push dword 2
push dword salto
push dword [handleConsola]
call _WriteConsoleA@20
mov ax, word[vocales]
mov bx, 10
mov cx, 0
mov dx, 0
mov edi, vocalesStr
enteroACadena:
mov bx, 10
div bx
add dx, '0'
push dx
inc cx
mov dx, 0
cmp ax, 0
jne enteroACadena
guardarDigitos:
pop dx
mov [edi], dx
inc edi
dec cx
cmp cx, 0
jne guardarDigitos
imprimirVocales:
push dword-11
xor eax, eax
call _GetStdHandle@4
mov [handleConsola], eax
xor eax, eax
mov eax, 0d
mov [ultimoArgumento], eax
push dword [ultimoArgumento], eax
push dword caractEscritos
push dword 19
push dword voca
push dword [handleConsola]
call _WriteConsoleA@20
push dword-11
xor eax, eax
call _GetStdHandle@4
mov [handleConsola], eax
xor eax, eax

```

```

mov eax, 0d
mov [ultimoArgumento], eax
push dword [ultimoArgumento]
push dword caractEscritos
push dword 4
push dword vocalesStr
push dword [handleConsola]
call _WriteConsoleA@20
salida:
push dword 0
call _ExitProcess@4

```

```

C:\Users\vbcsl>nasm -f win32 -o inversaw.obj inversaw.asm

C:\Users\vbcsl>ld inversaw.obj -m i386pe -e _main -L "C:\Windows\SysWOW64" -l kernel32 -o inversa.exe
ld: warning: resolving _GetStdHandle@4 by linking to _GetStdHandle
Use --enable-stdcall-fixup to disable these warnings
Use --disable-stdcall-fixup to disable these fixups
ld: warning: resolving _WriteConsoleA@20 by linking to _WriteConsoleA
ld: warning: resolving _ReadConsoleA@20 by linking to _ReadConsoleA
ld: warning: resolving _ExitProcess@4 by linking to _ExitProcess

C:\Users\vbcsl>inversa.exe
Digite 5 cadenas:
Cadena :h
Cadena :d
Cadena :s
Cadena :e
Cadena :s
Cadena concatenada :hdses
Cadena inversa :sesdh
Cantidad vocales :1
C:\Users\vbcsl>inversa.exe
Digite 5 cadenas:
Cadena :a
Cadena :e
Cadena :j
Cadena :i
Cadena :l
Cadena concatenada :aejil
Cadena inversa :lijea
Cantidad vocales :3

```

Calculadoraw.amd

```
segment .data
    opc1 db '1.- Operacion Suma',0xa
    opc2 db '2.- Operacion Resta',0xa
    opc3 db '3.- Operacion Division',0xa
    opc4 db '4.- Operacion Multiplicacion',0xa
    opc db 'Opcion :',0
    soln1 db 'Numero 1 :',0
    soln2 db 'Numero 2 :',0
    nega db '-'
    notneg equ ($-nega)
    suma db 'Suma',10
    notsum equ ($-suma)
    resu db 'Resultado :',0
segment .bss
    longitud resd 1
    handleConsolea resd 1
    caractEscritos resd 1
    ultimoArgumento resd 1
    opcion resb 3
    num1 resb 50
    num2 resb 50
    esperadoDigito resb 100
    puntero resb 8
segment .text
global _main
extern _GetStdHandle@4
extern _ReadConsoleA@20
extern _WriteConsoleA@20
extern _ExitProcess@4
_main:
    push dword -11
    xor eax,eax
    call _GetStdHandle@4
    mov [handleConsolea],eax
    xor eax,eax
    mov eax,0d
    mov [ultimoArgumento],eax
    push dword [ultimoArgumento]
    push dword caractEscritos
    push dword 19
    push dword opc1
    push dword [handleConsolea]
    call _WriteConsoleA@20
    push dword -11
    xor eax,eax
    call _GetStdHandle@4
    mov [handleConsolea],eax
    xor eax,eax
    mov eax,0d
    mov [ultimoArgumento],eax
    push dword [ultimoArgumento]
    push dword caractEscritos
    push dword 29
    push dword opc4
    push dword [handleConsolea]
    call _WriteConsoleA@20
    push dword -11
    xor eax,eax
    call _GetStdHandle@4
    mov [handleConsolea],eax
    xor eax,eax
    mov eax,0d
    mov [ultimoArgumento],eax
    push dword [ultimoArgumento]
    push dword caractEscritos
    push dword 9
    push dword opc
    push dword [handleConsolea]
    call _WriteConsoleA@20
    push dword -10
    push dword caractEscritos
    call _GetStdHandle@4
    mov [handleConsolea],eax
    xor eax,eax
    mov eax,0d
    mov [ultimoArgumento],eax
    push dword [ultimoArgumento]
    push dword caractEscritos
    call _GetStdHandle@4
    mov [handleConsolea],eax
    xor eax,eax
    mov eax,0d
    mov [ultimoArgumento],eax
    push dword [ultimoArgumento]
    push dword -10
    push dword caractEscritos
    call _GetStdHandle@4
```

```
mov [handleConsolea],eax
xor eax,eax
mov eax,2
mov [longitud],eax
xor eax,eax
mov eax,0d
mov [ultimoArgumento],eax
push dword [ultimoArgumento]
push dword caractEscritos
push dword 3
push dword opcion
push dword [handleConsolea]
call _ReadConsoleA@20
push dword -11
xor eax,eax
call _GetStdHandle@4
mov [handleConsolea],eax
xor eax,eax
mov eax,0d
mov [ultimoArgumento],eax
push dword [ultimoArgumento]
push dword caractEscritos
push dword 11
push dword soln1
push dword [handleConsolea]
call _WriteConsoleA@20
push dword -10
call _GetStdHandle@4
mov [handleConsolea],eax
xor eax,eax
mov eax,50
mov [longitud],eax
xor eax,eax
mov eax,0d
mov [ultimoArgumento],eax
push dword [ultimoArgumento]
push dword caractEscritos
push dword 50
push dword num2
push dword [handleConsolea]
call _ReadConsoleA@20
sub byte [opcion],0
cmp byte [opcion],byte 1
je operacionSuma
cmp byte [opcion],byte 2
je operacionResta
cmp byte [opcion],byte 3
je operacionDivision
cmp byte [opcion],byte 4
je operacionMultiplicacion
operacionSuma:
    push dword -11
    xor eax,eax
    call _GetStdHandle@4
    mov [handleConsolea],eax
    xor eax,eax
    mov eax,0d
    mov [ultimoArgumento],eax
    push dword [ultimoArgumento]
    push dword caractEscritos
    push dword 11
    push dword resu
    push dword [handleConsolea]
    call _WriteConsoleA@20
    mov eax, num1
    call atoi
    push eax
    mov eax, num2
```

<pre> call atoi pop edx add eax,edx call imprimir xor eax,eax mov eax,0d mov [ultimoArgumento],eax push dword [ultimoArgumento] call _ExitProcess@4 operacionResta: push dword-11 xor eax,eax call _GetStdHandle@4 mov [handleConsola],eax xor eax,eax mov eax,0d mov [ultimoArgumento],eax push dword [ultimoArgumento] push dword caractEscritos push dword 11 push dword resu push dword [handleConsola] call _WriteConsoleA@20 mov eax, num1 call atoi push eax mov eax, num2 call atoi pop edx cmp eax,edx jg signo sub edx,eax </pre>	<pre> mov eax, edx call imprimir xor eax,eax mov eax,0d mov [ultimoArgumento],eax push dword [ultimoArgumento] call _ExitProcess@4 signo: push eax push edx push dword-11 call _GetStdHandle@4 mov [handleConsola],eax xor eax,eax mov eax,0d mov [ultimoArgumento],eax push dword [ultimoArgumento] push dword caractEscritos push dword notneg push dword nega push dword [handleConsola] call _WriteConsoleA@20 pop edx pop eax sub eax,edx call imprimir xor eax,eax mov eax,0d mov [ultimoArgumento],eax push dword [ultimoArgumento] call _ExitProcess@4 operacionDivision: </pre>	<pre> push dword-11 xor eax,eax call _GetStdHandle@4 mov [handleConsola],eax xor eax,eax mov eax,0d mov [ultimoArgumento],eax push dword [ultimoArgumento] push dword caractEscritos push dword 11 push dword resu push dword [handleConsola] call _WriteConsoleA@20 mov eax, num1 call atoi push eax mov eax, num2 call atoi mov ebx,eax pop eax div ebx call imprimir xor eax,eax mov eax,0d mov [ultimoArgumento],eax push dword [ultimoArgumento] call _ExitProcess@4 operacionMultiplicacion: push dword-11 xor eax,eax call _GetStdHandle@4 mov [handleConsola],eax </pre>
---	--	--

<pre> mov [handleConsola],eax xor eax,eax mov eax,0d mov [ultimoArgumento],eax push dword [ultimoArgumento] push dword caractEscritos push dword 11 push dword resu push dword [handleConsola] call _WriteConsoleA@20 mov eax, num1 call atoi push eax mov eax, num2 call atoi mov ebx,eax pop eax mul ebx call imprimir xor eax,eax mov eax,0d mov [ultimoArgumento],eax push dword [ultimoArgumento] call _ExitProcess@4 atoi: mov esi, eax mov eax, 0 mov ecx, 0 .potencia10: xor ebx, ebx mov bl, [esi+ecx] cmp bl, 48 </pre>	<pre> jl .finatoi cmp bl, 57 jg .finatoi cmp bl, 10 je .finatoi cmp bl, 0 jz .finatoi sub bl, 48 add eax, ebx mov ebx, 10 mul ebx inc ecx jmp .potencia10 .finatoi: mov ebx, 10 div ebx ret imprimir: mov ecx, esperadoDigito mov ebx, 10 mov [ecx], ebx inc ecx mov [puntero], ecx bucle1: mov edx, 0 mov ebx, 10 div ebx add edx, 48 mov ecx, [puntero] mov [ecx], dl inc ecx mov [puntero], ecx </pre>	<pre> cmp eax, 0 jne bucle1 bucle2: mov ecx, [puntero] push dword-11 xor eax,eax call _GetStdHandle@4 mov [handleConsola],eax xor eax,eax mov eax,0d mov [ultimoArgumento],eax push dword [ultimoArgumento] push dword caractEscritos push dword 1 push dword ecx push dword [handleConsola] call _WriteConsoleA@20; mov ecx, [puntero] dec ecx mov [puntero], ecx cmp ecx, esperadoDigito jge bucle2 ret </pre>
---	---	--

```
C:\Users\vbcs1>nasm -f win32 -o calculadoraw.obj calculadoraw.asm

C:\Users\vbcs1>ld calculadoraw.obj -m i386pe -e _main -L "C:\Windows\SysWOW64" -l kernel32 -o calculadora.exe
ld: warning: resolving _GetStdHandle@4 by linking to _GetStdHandle
Use --enable-stdcall-fixup to disable these warnings
Use --disable-stdcall-fixup to disable these fixups
ld: warning: resolving _ReadConsoleA@20 by linking to _ReadConsoleA
ld: warning: resolving _WriteConsoleA@20 by linking to _WriteConsoleA
ld: warning: resolving _ExitProcess@4 by linking to _ExitProcess

C:\Users\vbcs1>calculadora.exe
1.- Operacion Suma
2.- Operacion Resta
3.- Operacion Division
4.- Operacion Multiplicacion
Opcion :2
Numero 1 :6
Numero 2 :10
Resultado :-4
```

```
C:\Users\vbcs1>calculadora.exe
1.- Operacion Suma
2.- Operacion Resta
3.- Operacion Division
4.- Operacion Multiplicacion
Opcion :1
Numero 1 :6
Numero 2 :7
Resultado :13
```

```
C:\Users\vbcs1>calculadora.exe
1.- Operacion Suma
2.- Operacion Resta
3.- Operacion Division
4.- Operacion Multiplicacion
Opcion :3
Numero 1 :20
Numero 2 :4
Resultado :5
```

```
C:\Users\vbcs1>calculadora.exe
1.- Operacion Suma
2.- Operacion Resta
3.- Operacion Division
4.- Operacion Multiplicacion
Opcion :4
Numero 1 :7
Numero 2 :6
Resultado :42
```

Conclusión

En esta práctica, profundizamos en la comprensión de cómo los sistemas operativos, como Windows y Linux, gestionan la interacción entre el hardware y el software. A través del uso del lenguaje ensamblador, pudimos ver cómo se ejecutan las instrucciones a nivel bajo, también entender la estructura interna y la lógica que manejan el funcionamiento de los sistemas operativos. Este enfoque nos permitió observar de cerca cómo se manejan procesos fundamentales como la asignación de memoria, la administración de dispositivos y el control del flujo de datos.

Trabajar con ambas plataformas nos brindó la oportunidad de comparar sus arquitecturas. **Windows**, con su enfoque en la interfaz gráfica y su amplio soporte para aplicaciones comerciales, demuestra ser una plataforma accesible para el uso cotidiano, pero también efectiva en la gestión de recursos. Por otro lado, **Linux**, conocido por su estabilidad y seguridad, demostró ser una herramienta poderosa para entornos de desarrollo y sistemas críticos, donde la capacidad de manipular cada aspecto del sistema es esencial.

El estudio profundo de los sistemas operativos no solo radica en su teoría, sino también en la capacidad de experimentar directamente cómo interactúan con el hardware y el software. A través del análisis práctico tanto de Windows como de Linux, es posible entender de manera tangible las diferencias clave entre estos dos sistemas operativos.

Algo a destacar es que el trabajo con lenguaje ensamblador en ambas plataformas permite desarrollar una mayor apreciación por el funcionamiento interno de los sistemas operativos. Al enfrentarse a la lógica y las estructuras de bajo nivel, se descubre cómo los sistemas operativos manejan las tareas básicas, desde la ejecución de procesos hasta la administración de memoria y el control de dispositivos. Este tipo de ejercicio ofrece no solo un reto técnico, sino también una oportunidad para profundizar en los principios fundamentales que hacen que los sistemas operativos modernos sean tan esenciales para la computación actual.

Al concluir con estas prácticas, queda claro que los sistemas operativos no son meras plataformas sobre las cuales se ejecutan programas, sino componentes complejos que gestionan los recursos de la manera más eficiente posible y haciendo que el usuario tenga una comodidad mayor en el funcionamiento de sus dispositivos.